



National Autonomous University of Mexico

Computer Engineering

Compilers

PARSER & SDT REPORT

TEAM 8

STUDENT:
321117524
321147561
321652485
321030122
321310275

Group:
5
Semester:
2027-I

Mexico City, Mexico. November 2026

Contents

1 Introduction **2**
1.1 Problem Statement . . . 2
1.2 Motivation . . . 2
1.3 Objectives . . . 2
1.4 Scope . . . 3

2 Theoretical Framework **3**
2.1 Parser . . . 3
2.2 Derivations and Parse Trees . . . 4
2.3 Ambiguity and Grammar Design . . . 4
2.4 Left Recursion . . . 4
2.5 Top-Down Parsing . . . 5
2.6 Table-Driven Predictive Parsing . . . 5
2.7 FIRST, FOLLOW, and the LL(1) Condition . . . 5
2.8 Bottom-Up Parsing . . . 5
2.9 Syntax Errors and Recovery . . . 6
2.10 Syntax-Directed Translation . . . 6
2.11 Symbol Table and Semantic Analysis . . . 6

3 Development **7**
3.1 Overview of the development . . . 7
3.2 Architecture and modules . . . 8
3.3 Context-Free Grammar . . . 10
3.4 FIRST and FOLLOW . . . 12
3.5 Predictive parsing table and LL(1) verification . . . 14
3.6 Predictive parser . . . 15
3.7 Parse tree . . . 17
3.8 SDT and symbol table . . . 18
3.9 Why Top-Down . . . 20

4 Results **21**
4.1 Test environment . . . 21
4.2 Correct programs . . . 21
4.3 Semantic errors . . . 25
4.4 Syntax errors . . . 26
4.5 Lexical errors . . . 27
4.6 Summary of results . . . 28

5 Conclusions **28**

A Names of the non-terminals **29**

B Productions of the grammar **29**

1 Introduction

1.1 Problem Statement

The lexical analyzer developed in the previous project converts the source text into classified tokens, but it does not examine how those tokens are arranged. A sequence of valid tokens can still be an invalid program, either because it breaks the structural rules of the language or because it expresses something that makes no sense. Detecting these problems is the task of the following two phases of the compiler front end: syntax analysis and semantic analysis.
This project addresses that need for the same subset of the C language handled by the lexer. A parser must take the stream of tokens, check it against a context-free grammar that defines the language, and build a parse tree that captures the syntactic structure of the program. On top of it, a Syntax-Directed Translation (SDT) must verify the semantic rules that a grammar cannot express; in this project, that an identifier is not declared twice in the same scope. The analysis must report clearly whether the program is correct or where it fails.

1.2 Motivation

Lexical analysis only recognizes words, while programming languages are built from nested structures such as expressions, statements, and blocks. The parser is the phase that discovers this hierarchy, and the grammar is the tool that defines it in a precise and unambiguous way. Implementing a parser therefore gives a deeper understanding of how a language is specified and how its constructs relate to each other.
Furthermore, the SDT allows the compiler to start giving meaning to the program. By attaching semantic rules to the structure of the tree, errors such as repeated declarations are detected during the analysis itself, and the parse tree produced in this process is the representation on which later phases will work. The project also shows how the phases of a compiler cooperate, and it applies concepts studied in class: grammars, derivations, FIRST and FOLLOW sets, and attributes.

1.3 Objectives

The main objective is to develop a parser that validates the structure of a program written in a subset of C, builds its parse tree, and verifies the semantic rules of its declarations through Syntax-Directed Translation. The specific objectives are:

- Define an unambiguous context-free grammar for the subset of C recognized by the lexer, adapted to LL(1) parsing through left factoring and the elimination of left recursion.
- Compute the FIRST and FOLLOW sets of every non-terminal and build the predictive parsing table, verifying that no cell receives more than one production.

- Implement a table-driven predictive parser that consumes the tokens produced by the lexer, validates the syntactic structure, and reports syntax errors with their line.
- Build a parse tree during the analysis.
- Implement SDT rules supported by a symbol table that detects identifiers declared twice in the same scope and validates the types of initializations.
- Integrate the parser with the lexer and report the outcome of both the syntactic and the semantic analysis.

1.4 Scope

The parser analyzes the same subset of C handled by the lexer: global and local variable declarations with optional initialization (`int`, `float`, `char` and `string`), function definitions with parameters and a return type (including `void`), function calls, chained assignments, `if-else`, `for` and `while` statements, nested blocks, `return` with a value, and arithmetic expressions with unary operators. Conditions may combine comparisons with `&&`, `||` and `!`.
The semantic analysis traverses the parse tree with a symbol table and a stack of scopes. The global scope exists from the beginning; every function (its parameters and its body share one scope), every block and every for statement opens a new one, which is closed when the construct ends. It verifies that a name is not declared twice in the same scope, stores the declared type of every variable and parameter, rejects identifiers used in an initialization without being declared, checks that the type of each initialization is compatible with the declared type, and requires numeric operands in the arithmetic and unary operators of those initializations. A name declared in an inner scope may hide one of an outer scope. The use of identifiers in assignments that are not initializations, in conditions, in call arguments and in return statements, the existence of called functions, and the type of returned values are not verified and are left as future work. The analysis stops at the first syntax error and at the first semantic error. Arrays, pointers, input and output, preprocessor directives, return without a value, and code generation are outside the scope of this project.

2 Theoretical Framework

2.1 Parser

Syntax analysis decides whether the sequence of tokens delivered by the lexer belongs to the language defined by the context-free grammar proposed in the previous project, and it exposes the structure that justifies that decision. The parser requests tokens one at a time, reports the position of any violation of the grammar, and hands a structured representation of the program to the semantic phases. Parsing methods that accept any grammar exist, but they are too slow for compilers, so practical

parsers are restricted to subclasses of grammars, mainly the LL and LR classes, and are organized as top-down or bottom-up methods [1]. This section presents the concepts needed to prepare the grammar, to build the parser, and to attach the semantic analysis to it.

2.2 Derivations and Parse Trees

A derivation is a sequence of steps in which a non-terminal is replaced by the body of one of its productions, starting from the start symbol and ending in a string of terminals. If the leftmost non-terminal is always replaced, the derivation is leftmost; if the rightmost one is replaced, it is rightmost. A parse tree is the graphical form of a derivation that ignores the order of the replacements: its root is the start symbol, its interior nodes are non-terminals, and its leaves, read from left to right, form the derived string. The importance of the parse tree lies in that it explicitly shows the hierarchical and grammatical structure of the program, allowing the compiler to understand how constructs are syntactically nested and related. Every parse tree corresponds to exactly one leftmost and one rightmost derivation, which is why parsers are described in terms of either of them.

2.3 Ambiguity and Grammar Design

A grammar is ambiguous when some string has more than one parse tree. Since the tree determines how the program is interpreted, ambiguity prevents most parsers from deciding the meaning of a construct. Two techniques remove it. The first encodes operator precedence and associativity in the structure of the grammar, assigning one non-terminal to each level of precedence. The second resolves the dangling-else problem by separating the statements whose `else` has already been matched from those that remain open, so that each `else` is paired with the closest `if`. The grammar of the previous project already applies both techniques.

2.4 Left Recursion

A non-terminal is left recursive when it can derive a string that begins with itself, as in the productions $A \to A\alpha \mid \beta$. A top-down parser cannot work with such productions, because it would call the procedure of A indefinitely without consuming any input. Bottom-up parsers, in contrast, handle left recursion without difficulty. For top-down parsing, the productions are rewritten as $A \to \beta A'$ and $A' \to \alpha A' \mid \varepsilon$, which generate the same language. The new grammar produces trees that lean to the right, so the attributes of the translation must be arranged to preserve the left associativity of the operators in the final representation. This rule removes only immediate left recursion; recursion through several non-terminals requires ordering them and substituting productions first. Together with the left factoring presented in the previous report, it is what prepares a grammar for predictive parsing.

2.5 Top-Down Parsing

A top-down parser builds the parse tree from the root toward the leaves, which is equivalent to finding a leftmost derivation of the input. Its central problem is choosing, for each non-terminal, the production to apply. In a recursive-descent parser, each non-terminal is implemented as a procedure that matches the terminals of the chosen production against the input and calls the procedures of the non-terminals it contains. In its general form this method may require backtracking. A predictive parser avoids it by choosing the production with a fixed number of lookahead tokens, and it can be written either as a set of recursive procedures or as a loop driven by a stack and a parsing table.

2.6 Table-Driven Predictive Parsing

A predictive parser can be implemented without recursion by using an explicit stack and the parsing table. The stack starts with the end marker \$ at the bottom and the start symbol on top. At each step the parser looks at the symbol on top of the stack and at the current token. If the symbol is a terminal, it must be equal to the token, and then both are consumed. If it is a non-terminal A and the token is a, the parser reads the cell $M[A, a]$: the production found there is applied by replacing A with the symbols of its right side, pushed in reverse order so that the leftmost symbol is expanded first. An empty cell means a syntax error. The input is accepted when the end marker is reached in both the stack and the input. The productions that are applied, in order, form a leftmost derivation of the input [1].

2.7 FIRST, FOLLOW, and the LL(1) Condition

The choice of production is based on two sets associated with the grammar. FIRST(α) is the set of terminals that can begin a string derived from α, and it includes ε when α can derive the empty string. FOLLOW(A) is the set of terminals that can appear immediately after the non-terminal A in some derivation, and it always contains the end marker \$ for the start symbol. A grammar is LL(1) when, for every pair of alternatives $A \to \alpha \mid \beta$, the sets FIRST(α) and FIRST(β) are disjoint and, if α can derive ε, FIRST(β) does not intersect FOLLOW(A). The name reflects the method: the input is read from left to right, a leftmost derivation is produced, and one token of lookahead is used. In the parsing table, each production is placed under the terminals of its FIRST set and, when it can derive ε, also under those of FOLLOW(A); the grammar is LL(1) exactly when no cell receives more than one production. No left-recursive or ambiguous grammar can satisfy this condition.

2.8 Bottom-Up Parsing

A bottom-up parser starts from the input and reduces it until reaching the start symbol, which traces a rightmost derivation in reverse. It is implemented as a shift-reduce parser with a stack: at each step it either shifts the next token onto the stack

or reduces a handle, that is, a substring on top of the stack that matches the body of a production and whose replacement is one step back in the derivation. The key decision is therefore when to reduce. When the available information does not determine the action, a shift/reduce or a reduce/reduce conflict appears. LR parsers resolve most of these decisions using tables built from an automaton of items; their variants (SLR, LALR, and canonical LR) differ in how precisely they use the lookahead, trading power for table size. These tables are normally generated by tools, and the class of grammars they accept is larger than the LL class and includes left-recursive grammars.

2.9 Syntax Errors and Recovery

Both LL and LR parsers detect a syntax error as soon as the portion of the input read so far cannot be completed into a valid program. The parser must report, at least, the token and the position where the error was detected, and the construct that was expected when it is possible to determine it. After reporting, it should recover in order to find further errors in the same execution without producing a flood of spurious messages. In panic-mode recovery, the parser discards tokens until it reaches a synchronizing token, usually a delimiter such as a semicolon or a closing brace; the FOLLOW sets are a natural source for these tokens in predictive parsing. Phrase-level recovery applies a small local correction, such as inserting a missing delimiter.

2.10 Syntax-Directed Translation

A Syntax-Directed Translation (SDT) associates attributes with the grammar symbols and semantic rules with the productions. An attribute is synthesized when it is computed from the children of a node, and inherited when it depends on the parent or the siblings of the node. The attributes of the terminals, such as the lexeme of an identifier or the value of a constant, are supplied by the lexer. When the rules are written as semantic actions inside the productions, each action is executed as soon as the symbols to its left have been recognized, so translation and checking take place during parsing without a separate pass. The type of attributes determines how naturally an SDT fits each method: definitions that use only synthesized attributes suit bottom-up parsing, where actions are executed at each reduction, while definitions in which information flows from above and from the left suit top-down parsing, where each procedure receives the inherited attributes as arguments and returns the synthesized ones.

2.11 Symbol Table and Semantic Analysis

The symbol table stores the information about the identifiers of the program, such as their name, type, and scope. Scopes are commonly handled with one table for each block, linked to the table of the enclosing block, and two basic operations: inserting a declaration in the current scope and looking up an identifier from the innermost scope outward. With this information, the semantic analysis performs the checks that the

grammar cannot express: an identifier must be declared before it is used, it cannot be declared twice in the same scope, and the types of operands, assignments, and conditions must be compatible with what their context expects. A syntax error is a failure of form that prevents the construction of the tree, whereas a semantic error appears when the structure is valid but its meaning is not.

3 Development

3.1 Overview of the development

The development followed the order in which the theory is presented in class. First, the grammar for the subset of C was designed and adapted so that a predictive parser could use it: left recursion was removed and the common prefixes were factored. Then FIRST and FOLLOW were computed for every non-terminal and used to fill the predictive parsing table. With the table ready, the parser was implemented as a loop driven by a stack. Finally, a parse tree is built during the analysis and the semantic rules of the SDT, which verify the declarations, are checked on it with the help of a symbol table.

Everything was written in C++, and the language that is analyzed is a subset of C, described by the grammar presented later in this section. The parser and the SDT were built directly from the concepts studied in the course (FIRST and FOLLOW sets, the predictive table, the stack algorithm, and attributes), so no parser generator is involved. The lexer is still the one generated with re2c in the previous project.

The lexer needed a few changes to work as the input of the parser. It now recognizes the keywords `void`, `string`, `for` and `while`, and the operators `<`, `>`, `!=`, `&&`, `||` and `!`. Every token also stores the line where it was found, so the later phases can report it. The end of the input is marked with a special `TOKEN_EOF` token whose lexeme is \$, which lets the parser know where the program ends. A small function translates each token into the name of the terminal used in the grammar; for example, a keyword token with the lexeme `int` becomes `KEYWORD_INT`.

Figure 1 shows how the phases are connected. The lexer sends the tokens to the parser, which uses the table built from FIRST and FOLLOW to validate them. If the syntax is correct, the SDT is applied to the resulting tree.

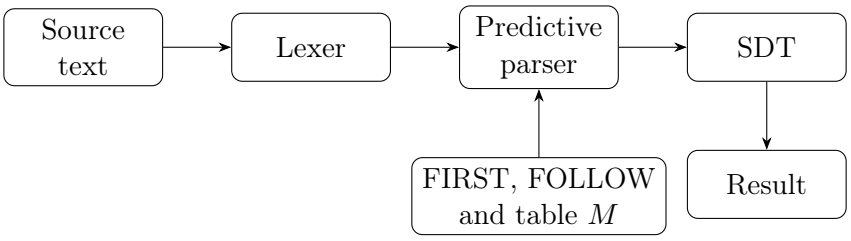


Figure 1: General flow of the analysis.

Depending on the input, the program ends in one of the four situations shown in Table 1. A lexical error stops the process before the parser starts, and a syntax error stops it before the semantic rules are checked, since there is no complete tree to evaluate.

Situation	**Cause**	**Output**
Lexical error	The lexer produces at least one error token	The lexical errors are reported with their line and the parser is not executed
Syntax error	No production in the table matches the current token	`Parsing error...` `SDT error...`
Semantic error	The syntax is valid but a rule of the SDT fails	`Parsing Success!` `SDT error...`
Correct program	The syntax is valid and all the rules of the SDT hold	`Parsing Success!` `SDT Verified!`

Table 1: Possible outcomes of the analysis.

3.2 Architecture and modules

The application is divided into modules, each one with a single responsibility, so that the lexical, syntactic and semantic phases can be modified without touching the others. Table 2 describes each file.

Module	Responsibility
`lexer.re` and `lexer.cpp`	Specification of the scanner and the C++ code generated from it with re2c.
`lexer_utils`	Loop that calls the scanner, completes the subtype of every token and reports the lexical errors with their line.
`token`	Token types and subtypes, and the function that converts a token into the terminal symbol used by the grammar.
`grammar`	Stores the 95 numbered productions, the 46 non-terminals, the 34 terminals and the start symbol.
`first_follow`	Computes the FIRST and FOLLOW sets by iterating until no set changes.
`parsing_table`	Builds the predictive table and stops with a message if a cell receives two different productions.
`parse_tree`	Defines the nodes of the parse tree.
`parser`	Stack algorithm, syntax error messages, optional trace of the productions and construction of the parse tree.
`symbol_table`	Symbol table implemented with `map<string, Symbol>`; each `Symbol` stores the name, the type and the line of a variable. `initializeSymbolTable` clears it and `declareSymbol` rejects a name that already exists in it. The scopes are handled by the SDT on top of this table.
`sdt`	Traversal of the parse tree with a context that holds the visible variables, the names declared in each open scope and the tables of the enclosing scopes. It registers global and local variables, parameters and for declarations, opens and closes scopes for functions, blocks and for, infers the types of initializations, and rejects undeclared identifiers, non-numeric operands, incompatible initializations and chained assignments inside an initialization.
`main.cpp`	Reads the input file, calls each phase in order and prints the result.

Table 2: Modules of the application.

The main program reads the input from a text file and calls the lexer. If the lexer reports errors, they are printed with their line and the process ends. Otherwise the tokens are given to the parser, which loads the grammar, calculates FIRST and FOLLOW, builds the table and runs the stack algorithm, returning the parse tree if the syntax is correct. Only then the semantic analysis is executed on the tree, using the list of tokens to recover the lexemes and lines of the identifiers. The project is kept in a public repository that includes the instructions to generate the scanner with re2c and to compile all the files together, the same procedure used in the previous project.

3.3 Context-Free Grammar

The parser is driven by an unambiguous context-free grammar (CFG) designed to recognize a subset of the C programming language. The language encompasses global and local variable declarations with optional initialization, function definitions and calls, chained assignments, conditional statements (`if/else`), iterative structures (`for` and `while`), nested code blocks, return statements, and arithmetic expressions with standard operator precedence.

Formally, as defined in the grammar data structure, the grammar consists of 46 non-terminal symbols, 34 terminal symbols (the lexer tokens plus `TOKEN_EOF`), and 95 numbered production rules, with `Program` as the start symbol. Two of those non-terminals, `FunctionType` and `FunctionDefinition` (rules 21, 22 and 88), are auxiliary definitions that cannot be reached from the start symbol, because `GlobalElement` takes their place; the effective grammar therefore has 44 non-terminals and 92 productions. The program uses Spanish names for the non-terminals; this report uses English names, and Appendix A gives the correspondence. The complete list of productions is in Appendix B. To make the grammar suitable for LL(1) predictive parsing, left recursion was eliminated and left factoring was applied.

Elimination of Left Recursion and Operator Precedence

Top-down parsers cannot handle left-recursive productions, so they were rewritten into right-recursive forms using auxiliary "Prime" non-terminals (e.g., `ExpressionPrime`, `TermPrime`). Operator precedence is encoded by assigning a specific non-terminal to each level of the expression hierarchy, from logical operators down to primary elements. Because this elimination process generates right-leaning parse trees, a key requirement for the Syntax-Directed Translation (SDT) is that it must arrange its attributes to correctly preserve the original left associativity of the operators during semantic evaluation.

Left Factoring and Disambiguation

Left factoring was applied to resolve FIRST/FIRST conflicts where multiple alternatives shared a common prefix. Table 3 summarizes the most relevant transformations applied to achieve the definitive LL(1) grammar.

Construct	**Preliminary Form / Conflict**	**Definitive Factored Form**
Variable List	VariableList → Variable | Variable COMMA VariableList... (Conflict: FIRST/FIRST)	VariableList → Variable VariableListPrime VariableListPrime → COMMA Variable ... | ε
Argument List	ArgumentList → Expression | Expression COMMA ArgumentList... (Conflict: FIRST/FIRST)	ArgumentList → Expression ArgumentListPrime | ε ArgumentListPrime → COMMA Expression ... | ε
Function Call	Primary → IDENTIFIER | FunctionCall (Conflict: FIRST/FIRST)	Primary → IDENTIFIER FunctionCallPrime FunctionCallPrime → LPAREN ArgumentList RPAREN | ε
Assignment vs. Expression	Assignment → IDENTIFIER OPERATOR_ASSIGN Assignment | Expression (Conflict: FIRST/FIRST)	Assignment → IDENTIFIER ContinueAssignment | StartWithoutId ContinueExpression
Global Element	GlobalElement → Declaration | FunctionDefinition (Conflict: FIRST/FIRST on Types)	GlobalElement → Type IDENTIFIER GlobalContinuation GlobalContinuation → LPAREN OptionalParameters RPAREN Block | OptionalInitialization VariableListPrime SEMICOLON

Table 3: Summary of the left factoring transformations applied to resolve FIRST/FIRST conflicts.

A key result of this factoring is the non-terminal ContinueAssignment (rules 43 and 44), which cleanly distinguishes between an assignment and a standalone expression after an identifier has been consumed. The parser makes this decision using a single token of lookahead. If the next token is OPERATOR_ASSIGN (=), rule 43 is selected, and the parser recursively parses an assignment. If the next token is anything else that can start an expression continuation (such as arithmetic operators, a parenthesis for a function call, or a token in its FOLLOW set), rule 44 is selected to parse the rest of the arithmetic or logical expression.

Unambiguity and LL(1) Verification

The absence of ambiguity is structurally guaranteed by the LL(1) property. During the construction of the predictive parsing table, the program verifies that no cell $M[A, a]$ contains more than one production. For the 95 productions and 46 non-terminals of the grammar, no FIRST/FIRST or FIRST/FOLLOW conflicts were detected. This ensures the parser deterministically chooses the correct production for any reachable rule with a single token of lookahead.

Language-Specific Design Decisions

Several specific decisions define the recognized subset of C:

- **Statements and Assignments:** The non-terminal `Assignment` recognizes both standard assignments (e.g., `a = 10;`) and standalone expressions (e.g., `a + 4;`).
- **Chained Assignments:** Because initialization relies on `Assignment`, syntactically valid chains like `int a = b = 10;` are accepted by the parser and left to the SDT for semantic validation.
- **Dangling-Else Avoidance:** The grammar inherently avoids this ambiguity because both `if` and `else` branches strictly require a `Block` (delimited by braces). Thus, an `else` cannot ambiguously attach to the nearest `if`.
- **Conditions and expressions:** Comparisons and the logical operators only appear inside the conditions of `if`, `while` and `for`, so a statement such as `x = a > b;` is not part of the language. The operator `!` must be followed by a comparison between parentheses.

3.4 FIRST and FOLLOW

The predictive parser relies on two sets associated with each symbol of the grammar to know which production to use.

FIRST(X): the set of terminals that can appear at the beginning of the strings derived from X. If X can derive the empty string ε, then ε is also in FIRST(X).

FOLLOW(X): the set of terminals that can appear immediately after the non-terminal X in some derivation. By definition, the end-of-input marker `TOKEN_EOF` is in FOLLOW(S) for the start symbol S = `Program`.

Both sets are calculated by an iterative algorithm that stops when a complete pass does not change any set.

To calculate FIRST, the algorithm starts with an empty set for all non-terminals and applies the productions repeatedly. For a production $A \to X_1 X_2 \dots X_n$, the terminals of FIRST(X_1) (excluding ε) are added to FIRST(A); if X_1 can derive ε, the terminals of FIRST(X_2) are added, and so on. If every symbol on the right-hand side can derive ε, then ε is added to FIRST(A). The same procedure, applied to a sequence of symbols, is reused later to build the parsing table.

For FOLLOW, the algorithm initializes FOLLOW(`Program`) = {`TOKEN_EOF`} and goes through all the productions. For each occurrence of a non-terminal B in a production $A \to \alpha B \beta$, the terminals of FIRST(β) (excluding ε) are added to FOLLOW(B); if β can derive ε (or is empty), then all terminals of FOLLOW(A) are also added to FOLLOW(B). The process repeats until no set changes.

Table 4 shows the sets of the most representative non-terminals, which are the ones that decide the interesting cases of the parser. Here `ID`, `CONST` and `LIT` stand for identifier, constant and literal, and `$` is the end of the input. The sets of all the non-terminals are produced by the program when the table is built.

Non-terminal	**FIRST**	**FOLLOW**
Program	`int`, `float`, `char`, `string`, `void`, ε	`$`
GlobalElement	`int`, `float`, `char`, `string`, `void`	`int`, `float`, `char`, `string`, `void`, `$`
GlobalContinuation	`(`, `=`, `,`, `;`	`int`, `float`, `char`, `string`, `void`, `$`
Type	`int`, `float`, `char`, `string`	`ID`
VariableListPrime	`,`, ε	`;`
Expression	`ID`, `CONST`, `LIT`, `(`, `+`, `-`	`)`, `,`, `;`, `==`, `>=`, `<=`, `>`, `<`, `!=`, `&&`, `||`
ExpressionPrime	`+`, `-`, ε	`)`, `,`, `;`, `==`, `>=`, `<=`, `>`, `<`, `!=`, `&&`, `||`
TermPrime	`*`, `/`, ε	`+`, `-`, `)`, `,`, `;`, `==`, `>=`, `<=`, `>`, `<`, `!=`, `&&`, `||`
Primary	`ID`, `CONST`, `LIT`, `(`	`*`, `/`, `+`, `-`, `)`, `,`, `;`, `==`, `>=`, `<=`, `>`, `<`, `!=`, `&&`, `||`
FunctionCallPrime	`(`, ε	`*`, `/`, `+`, `-`, `)`, `,`, `;`, `==`, `>=`, `<=`, `>`, `<`, `!=`, `&&`, `||`
Assignment	`ID`, `CONST`, `LIT`, `(`, `+`, `-`	`;`, `,`, `)`
ContinueAssignment	`=`, `(`, `*`, `/`, `+`, `-`, ε	`;`, `,`, `)`
ContinueExpression	`*`, `/`, `+`, `-`, ε	`;`, `,`, `)`
OptionalComparison	`==`, `>=`, `<=`, `>`, `<`, `!=`, ε	`&&`, `||`, `)`, `;`
StatementList	`int`, `float`, `char`, `string`, `ID`, `CONST`, `LIT`, `(`, `+`, `-`, `return`, `if`, `for`, `while`, `{`, `;`, ε	`}`
OptionalElse	`else`, ε	`int`, `float`, `char`, `string`, `ID`, `CONST`, `LIT`, `(`, `+`, `-`, `return`, `if`, `for`, `while`, `{`, `;`, `}`

Table 4: FIRST and FOLLOW sets of the main non-terminals.

Some of them are worth reading. FIRST of *Expression* and of *Assignment* are the same, since both can start with an identifier, a constant, a literal, an opening parenthesis or a sign. FIRST of *FunctionCallPrime* contains `(` and ε, and `(` does not appear in its FOLLOW, so seeing an opening parenthesis after an identifier always means a function call. Finally, FOLLOW of *ContinueAssignment* contains `;`, `,` and `)` but not `=`, which is what allows the parser to choose between rules 43 and 44.

3.5 Predictive parsing table and LL(1) verification

The predictive parsing table M has one row for each non-terminal and one column for each terminal. It is built production by production. For a production $A \to \alpha$, the production number is written in the cell $M[A, a]$ for every terminal a in FIRST(α) (excluding ε), and, if α can derive ε, also in the cells $M[A, b]$ for every terminal b in FOLLOW(A). Before writing a number, the program checks whether the cell already has a different production; if so, it stops with a message that names the cell and the two rules in conflict. Building the table with the 95 productions of the grammar did not trigger that message, so no cell has more than one production and the grammar is LL(1).

The concrete implementation in `parsing_table.h` uses a global table `map<string, map<string, int» table`. `createPredictiveTable()` iterates over the productions, computes `FIRST` of the right-hand side with `firstOfSequence`, excludes ε from the columns, and, if the production can derive the empty string, also adds the terminals in `FOLLOW(production.left)`. Before writing a cell, it checks whether the cell already contains a different production; if so, it throws a `runtime_error` naming $M[A, a]$ and the two rules in conflict. `printParsingTable()` prints every entry as `M[non-terminal, terminal] = production number`.
Table 5 shows the filled cells of some rows. Each entry reads as "production number: columns where it is written".

Non-terminal	Productions and columns
Program	90: `int`, `float`, `char`, `string`, `void` 91: `$`
GlobalElement	92: `int`, `float`, `char`, `string` 93: `void`
GlobalContinuation	94: `(` 95: `=`, `,`, `;`
OptionalInitialization	5: `=` 6: `,`, `;`
VariableListPrime	2: `,` 3: `;`
Assignment	41: `ID` 42: `CONST`, `LIT`, `(`, `+`, `-`
ContinueAssignment	43: `=` 44: `(`, `*`, `/`, `+`, `-`, `;`, `,`, `)`
FunctionCallPrime	38: `(` 39: every terminal of its FOLLOW set
OptionalElse	80: `else` 81: every terminal of its FOLLOW set

Table 5: Selected cells of the predictive parsing table.

The verification can be followed by groups of productions, because in each group the alternatives of a non-terminal start with different terminals, or the empty alternative is selected only with terminals of FOLLOW that do not start any other alternative.

- **Lists (rules 1–16):** the alternatives of *VariableListPrime*, *ArgumentListPrime* and *ParameterListPrime* start with a comma or are empty, and the comma is not in their FOLLOW sets. *OptionalParameters* starts with a type or is empty, and its FOLLOW is only `)`.
- **Types (17–22):** each alternative starts with a different keyword.
- **Expressions (23–39):** *ExpressionPrime* and *TermPrime* start with operators that never appear in the FOLLOW set of the same non-terminal. For *Primary*,

the four alternatives start with `CONST`, `LIT`, `ID` and `(`. For *FunctionCallPrime*, `(` is not in FOLLOW.

- **Assignment (41–50):** the two alternatives of *Assignment* start with an identifier or with a different set of terminals. For *ContinueAssignment*, the first alternative starts with `=`, and neither the second one nor FOLLOW contain it.
- **Comparisons (51–67):** the six operators of *ComparisonOperator* are different terminals, and *LogicalComparison* chooses the second alternative only with `!`.
- **Blocks and statements (68–78):** the alternatives of *Statement* start with a type, one of the starts of *Assignment*, `return`, `if`, `for`, `while`, `{` or `;`, and these sets are pairwise disjoint.
- **Conditionals and loops (79–87):** `else` is not in the FOLLOW set of *OptionalElse*, because no statement can start with `else`. In *ForInitialization*, the alternatives start with a type or with the start of an assignment, and the empty one is selected with `;`.
- **Program (90–95):** *GlobalElement* separates the declarations from the `void` functions by the first terminal, and *GlobalContinuation* separates a function (`(`) from a declaration (`=`, `,`, `;`).

3.6 Predictive parser

The predictive parser implemented in this project is a non-recursive, table-driven LL(1) parser that uses an explicit stack to simulate the leftmost derivation of the input string. At each step, the parser consults the predictive parsing table to select the appropriate production rule based on the non-terminal at the top of the stack and the current input token.

Stack-Driven Parsing Algorithm

The parsing process follows a deterministic loop driven by the state of the stack and the token stream provided by the lexer. Before it starts, the list of tokens is checked looking for lexical errors, because the parser only works with valid tokens.

1. The stack is initialized by pushing the end-of-input marker (`TOKEN_EOF`) and the start symbol (`Program`).
2. The symbol at the top of the stack and the current input token (lookahead) are used to query the LL(1) predictive table.
3. If the top of the stack is a non-terminal, the table provides the corresponding production rule number. The parser retrieves this rule and pushes the symbols of its right-hand side onto the stack in reverse order. A production with an empty right side only removes the non-terminal from the stack.

4. If the top of the stack is a terminal, it must match the current input token. The token is consumed, the terminal is popped from the stack, and the parser advances to the next token.

5. The process continues until the end-of-input marker is successfully matched, indicating a successful parse. A syntax error is reported if no valid production exists in the predictive table or if the expected terminal does not match the current input token.

While the productions are applied, the nodes of the parse tree are created, as explained in Section 3.7. The productions applied, in the order they were used, are a leftmost derivation of the input, and the program can print them as a trace.

Table 6 shows the trace for the official example `int x = 10;`. The numbers refer to the productions of the grammar. At the end, after consuming the semicolon, the non-terminal *Program* is expanded to ε because the lookahead is the end marker, which belongs to its FOLLOW set, and then the end marker is matched.

Step	**Top of stack**	**Lookahead**	**Action**
1	*Program*	`int`	Rule 90: *GlobalElement Program*
2	*GlobalElement*	`int`	Rule 92: *Type* `ID` *GlobalContinuation*
3	*Type*	`int`	Rule 17: `int`
4	`int`	`int`	Match and consume
5	`ID`	`x`	Match and consume
6	*GlobalContinuation*	`=`	Rule 95: *OptionalInitialization VariableListPrime* `;`
7	*OptionalInitialization*	`=`	Rule 5: `=` *Assignment*
8	`=`	`=`	Match and consume
9	*Assignment*	`10`	Rule 42: *StartWithoutId ContinueExpression*
10	*StartWithoutId*	`10`	Rule 46: `CONST`
11	`CONST`	`10`	Match and consume
12	*ContinueExpression*	`;`	Rule 45: *TermPrime ExpressionPrime*
13	*TermPrime*	`;`	Rule 30: ε
14	*ExpressionPrime*	`;`	Rule 26: ε
15	*VariableListPrime*	`;`	Rule 3: ε
16	`;`	`;`	Match and consume
17	*Program*	`$`	Rule 91: ε
18	`$`	`$`	Match, input accepted

Table 6: Trace of the predictive parser for `int x = 10;`.

Since the table is built from the FIRST and FOLLOW sets, the parser detects an error as soon as the part of the input that has been read cannot be the beginning of a valid program. The message includes the line of the token where the problem was detected, the token that was found, the terminals that were expected, which are the columns with a production in the row of the non-terminal that was on top of the

stack, and that non-terminal as context. This also means that the reported line is not always the line where the mistake was made. If a semicolon is missing at the end of a line, for example, the error is found in the next token, which is usually on the following line. No recovery strategy was implemented, so the analysis stops at the first syntax error, and in that case the SDT is not executed.

3.7 Parse tree

The parse tree is built at the same time as the stack algorithm runs. Each node stores four things: the grammar symbol, the number of the production that was applied to it (zero if it is a terminal), the index of the token that it matched (only for terminals), and the list of its children. The tree is kept in a vector and the nodes refer to each other by their position, so the references remain valid while the tree grows.

The construction follows the stack. The root is a node for the start symbol, and every entry of the stack carries the position of its node. When a non-terminal is expanded with a production, one child is created for each symbol of the right side, the children are linked to the node, and they are pushed on the stack together with their positions. If the production is empty, the node receives a single child called `epsilon`. When a terminal is matched with a token, its node stores the index of that token, which is the way to recover its lexeme and its line later.

For the official example `int x = 10;`, the parser applies the productions 90, 92, 17, 95, 5, 42, 46, 45, 30, 26, 3 and 91, in that order. The resulting tree is shown in Figure 2. It is a concrete syntax tree: it keeps the auxiliary non-terminals and the ε nodes that complete the derivation, which is the main difference with an abstract syntax tree, and it is the structure that the semantic analysis traverses.

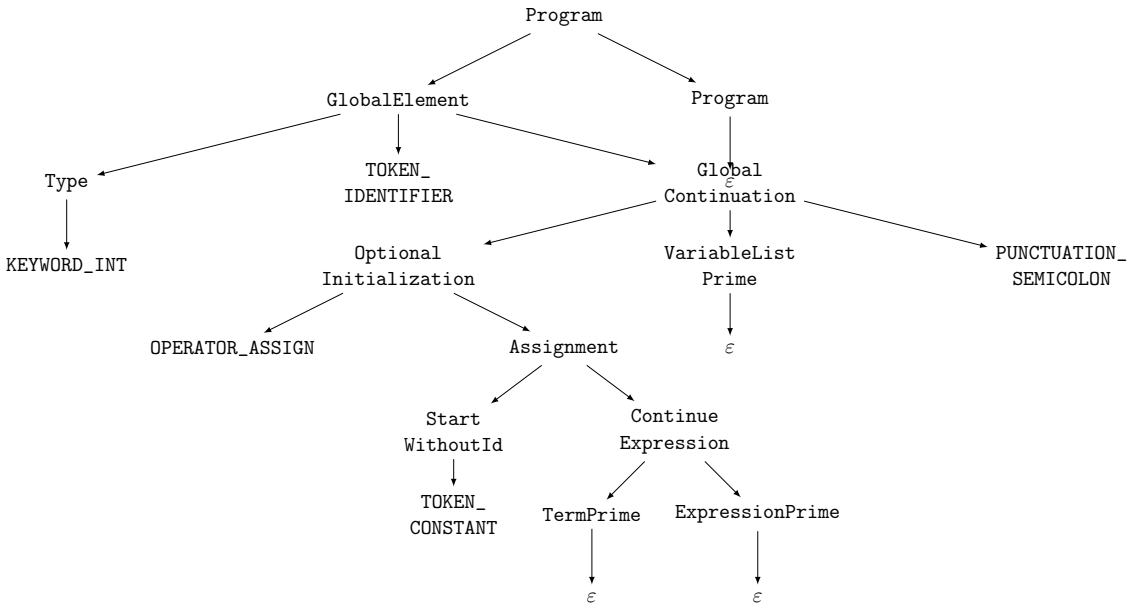


Figure 2: Parse tree built for the declaration `int x = 10;`.

3.8 SDT and symbol table

The grammar only describes the form of the programs. The rules that depend on context, such as the fact that a name cannot be declared twice in the same scope, cannot be expressed with a context-free grammar, and that is why a Syntax-Directed Translation is added.

The traversal is a single pass over the tree, in depth and from left to right, which is the order of the derivation. At each node the analysis identifies the construct through the number of the production stored in the node, so every semantic rule is attached to specific productions of the grammar. The attributes follow the L-attributed scheme: the declared type is read at the node of `Type` and used for every variable of the list that follows it (inherited), the type of an expression is computed from its operands (synthesized), and the lexeme and the line of an identifier are supplied by the lexer through the index of the token stored in the terminal node. Because the information only flows from parents and from left siblings, no second pass is needed.

The symbol table stores the name, declared type and line of each variable (Symbol); declareSymbol rejects a name that already exists in it. The scopes are handled by the traversal through a context that keeps the table of visible variables, the set of names declared in each open scope, and a copy of the table of every enclosing scope. Entering a scope saves a copy and opens an empty set of names; leaving it restores the copy. A name is rejected only if it is in the set of the current scope; if it exists in an outer scope it is accepted and hides it while the inner scope is open.

`verifySDT` initializes the context with the global scope and calls `analyzeNode` on the

root. At each node, `analyzeNode` identifies the construct through the number of its production and applies the rules of Table 7; for any other node it visits the children in order.

The type of an initialization is inferred by `inferExpressionType`, which applies these rules:

- A constant is `int` or `float` according to its lexeme, and a literal is `string`.
- An identifier takes the type stored in the table of visible variables, which includes the outer scopes and the parameters; if it is not there, an undeclared-identifier error is reported.
- Arithmetic and unary operators require numeric operands.
- A parenthesized expression has the type of the inner expression.
- A function call has an unknown type and is accepted without checking it; a chained assignment inside an initialization is reported as unsupported.

Then `compatibleTypes` decides whether the value can be stored: it accepts the same type and lets a `float` variable receive an `int`, but an `int` variable cannot receive a `float`.

Table 7 lists the rules that the traversal applies and the ones that it leaves out.

Where	**Action**
Global variable (rules 92 and 95)	The first variable is registered with the declared type and its initialization is validated; then the variable list is traversed and every variable is registered and validated.
Function (rules 92 with 94, and 93)	A scope is opened, the parameters are registered, the body is analyzed in that same scope and the scope is closed. The function name is not registered.
Local declaration (rule 40), declaration in a `for` (rule 83)	The variables are registered in the current scope and their initializations are validated.
`for` (rule 82), block (rule 68)	Each one opens a scope and closes it when it ends.
Repeated name	Rejected if the name is already declared in the current scope; the error reports the line of the second declaration.
Assignments, conditions, calls, `return`	Visited, but no rule is applied: they are not verified.

Table 7: Semantic rules applied by the current SDT.

`Example: int x = 10;`. The parser applies the productions 90, 92, 17, 95, 5, 42, 46, 45, 30, 26, 3, and 91 (Section 3.7). The semantic analysis then proceeds as follows:

1. The traversal starts at `Program` (rule 90), which has no semantic rule, and reaches the global element (rule 92).

2. The node `Type` resolves to the lexeme of its terminal, `int`, and the identifier node resolves to `x`, with the line of its token.

3. The continuation is rule 95, so it is a variable declaration. `declareSymbol` inserts `x` with type `int` in the empty table, and the insertion succeeds.

4. The initialization is the constant `10`, whose type is `int`. Since `int` is compatible with `int`, the validation succeeds.

5. The variable list is empty (rule 3), and the last `Program` derives the empty string (rule 91). The output is `Parsing Success!` and `SDT Verified!`.

Table 8 shows how the same mechanism produces the errors of the tests of Section 4. In every case the parser succeeds, because the structure is valid, and the error is found by the SDT.

Input	**Table before**	**Result**
`int x = 10;`	empty	Inserted: `x`, `int`, line 1. Type of the value: `int`. Accepted.
`int x = 10; int x = 20;`	`x` (`int`)	The second insertion fails; semantic error on line 2 for the repeated declaration of `x`.
`int x = "Hola";`	empty	Inserted: `x`, `int`. Type of the value: `string`, not compatible with `int`; semantic error on line 1.

Table 8: Behavior of the symbol table and of the type check in three declarations.

The semantic verification covers declarations, scopes and the initializations of the declared variables. Assignments that are not initializations, conditions, call arguments and `return` statements are not checked, functions are not registered so their calls are not validated, and the type of a returned value is not compared with the type of its function. Extending the traversal to these constructs only requires applying the same lookup and type inference from their productions.

3.9 Why Top-Down

A top-down parser was chosen instead of a bottom-up one for three main reasons.

First, the grammar of the language can be made LL(1) with the transformations seen in class. After removing left recursion and factoring the common prefixes, the construction of the parsing table did not produce any conflict. Since the table comes directly from the FIRST and FOLLOW sets, building it is a mechanical process that can also be checked by hand.

Second, the implementation with a stack makes the derivation explicit. Each step of the analysis is a production or a match, so it is easy to print the trace, to follow the parser when something fails, and to explain it step by step.

Third, the SDT fits this method well. The semantic rules need information that goes from a parent to its children and from left to right, such as the declared type that must reach every variable of a list. This is an inherited attribute, and a definition that only uses inherited and synthesized attributes in that way is L-attributed, which is the kind that can be evaluated while traversing the tree built by a top-down analysis. In a bottom-up parser the actions are executed when a reduction is made, so passing information downwards would need extra mechanisms.

This choice has a cost. The grammar has to be rewritten to be LL(1), and the non-terminals added to remove left recursion produce trees that lean to the right, so the left associativity of the arithmetic operators has to be preserved when the attributes are evaluated. A bottom-up parser accepts a larger class of grammars, including left-recursive ones, but it needs an automaton of items and the ACTION and GOTO tables, which are much heavier to build by hand and were not necessary for this subset of C [1].

4 Results

The tests were designed to cover the four possible outcomes of the analysis: correct programs, programs with a semantic error, programs with a syntax error and programs with a lexical error. Each test shows the input, a capture of the output of the program, and a short explanation of why the program produces that result according to the theory presented before.

4.1 Test environment

The program was compiled with a C++ compiler together with the scanner generated by re2c, as in the previous project, and executed from a terminal. For each test, the input was written in the file read by the main program and the program was run without arguments.

4.2 Correct programs

Test 1: Official example.

```
int x = 10;
```

The declaration is accepted by the parser and by the semantic analysis. The input belongs to the language because the derivation of Table 6 reaches the end marker, and the variable is declared once, so no semantic rule fails.

Test 2: Program with the main constructs.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Parsing Success!

========== SEMANTIC ANALYSIS ==========
SDT Verified!
```

Figure 3: Output of Test 1.

```
int contador = 0;

int suma(int a, int b) {
    return a + b;
}

int main() {
    int total = suma(contador, 3);
    for (int i = 0; i < 5; i = i + 1) {
        total = total + i * 2;
    }
    while (total > 10 && contador < 3) {
        contador = contador + 1;
    }
    if (total == 10) {
        total = -total;
    } else {
        contador = 0;
    }
    return total;
}
```

This program combines a global variable, a function with parameters, a call, a `for` loop, a `while` loop with a logical condition, an `if-else` and a unary operator. The parser accepts all these constructs because they have productions in the grammar. The SDT registers `contador`, opens a scope for `suma` with its parameters, and in `main` registers `total` (its initializer is a call, so its type is unknown and accepted); the `for` and each block open their own scope. No name is repeated in a scope.

Test 3: Operator precedence.

```
int a = 1;
int b = 2;
int r = a + b * 3 - (a / 2);
```

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Parsing Success!

========== SEMANTIC ANALYSIS ==========
SDT Verified!
```

Figure 4: Output test 2.

The expression is accepted. The product and the division are produced by the non-terminals of the lower levels (*Term*), so they are grouped before the addition and the subtraction, and the parentheses force the grouping of the last term.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Parsing Success!

========== SEMANTIC ANALYSIS ==========
SDT Verified!
```

Figure 5: Output test 3.

Test 4: Nested conditionals.

```
int main() {
    int a = 1;
    int b = 2;
    int x = 0;
    if (a == 1) {
        if (b == 2) {
            x = 1;
        } else {
            x = 2;
        }
    }
    return x;
}
```

The `else` belongs to the inner `if` without any ambiguity, because both branches must be blocks between braces and there is a single way to derive the program. The SDT opens a scope for `main` and for each block; every variable is declared once, so the program is accepted.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Parsing Success!

========== SEMANTIC ANALYSIS ==========
SDT Verified!
```

Figure 6: Output of Test 4.

Test 5: Empty program. The input file is empty. The parser expands *Program* with rule 91, since the end marker belongs to its FOLLOW set, and the program is accepted with no declarations to verify.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Parsing Success!

========== SEMANTIC ANALYSIS ==========
SDT Verified!
```

Figure 7: Output Test 5.

Test 6: Same name in different scopes.

```
int x = 1;
int main() {
    int x = 2;
    return x;
}
```

The program is accepted. The second `x` is inside `main`, whose scope is new, so it hides the global `x` instead of repeating it; when the scope closes, the global one is visible again.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Parsing Success!

========== SEMANTIC ANALYSIS ==========
SDT Verified!
```

Figure 8: Output Test 6.

4.3 Semantic errors

Test 7: Repeated global declaration.

```
int x = 10;
int x = 20;
```

The syntax is valid, so the parser accepts the input and the tree is built, but the semantic analysis finds that `x` already exists in the global scope and reports the repeated declaration in line 2. This kind of rule depends on the context and cannot be expressed in the grammar, which is why it is detected after the syntactic analysis.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Parsing Success!

========== SEMANTIC ANALYSIS ==========
SDT error: Error semantico en linea 2: variable 'x' declarada previamente
```

Figure 9: Output Test 7.

Test 8: Type incompatibility in variable initialization.

```
int x = "Hola";
```

The variable `x` is declared as an integer but initialized with a string literal. Since the assigned value does not match the declared type, the semantic analyzer detects a type incompatibility and reports an error.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Parsing Success!

========== SEMANTIC ANALYSIS ==========
SDT error: Error semantico en linea 1: inicializacion incompatible de 'x': se esperaba int, se
 obtuvo string
```

Figure 10: Output Test 8.

4.4 Syntax errors

Test 9: Missing semicolon.

```
int x = 10;
int y = 5
int z = 1;
```

The semicolon is missing at the end of line 2, but the error is reported in line 3. After the constant `5`, the parser is expanding *ContinueExpression* and the next token is `int`, which has no production in that row of the table. This is the first token that cannot continue a valid program, and it is on the following line.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Error sintactico en linea 3: se encontro 'int' (KEYWORD_INT). Se esperaba: OPERATOR_DIV, OPERA
TOR_MINUS, OPERATOR_MULT, OPERATOR_PLUS, PUNCTUATION_COMMA, PUNCTUATION_RPAREN, PUNCTUATION_SE
MICOLON. Contexto: ContinuarExpresion
Parsing error
```

Figure 11: Output Test 9.

Test 10: Missing closing parenthesis.

```
int main( {
```

After the opening parenthesis, the parser expects a parameter or a closing parenthesis. The token `{` is not in the row of *OptionalParameters*, so the cell is empty and the error is reported in line 1.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Error sintactico en linea 1: se encontro '{' (PUNCTUATION_LBRACE). Se esperaba: KEYWORD_CHAR,
KEYWORD_FLOAT, KEYWORD_INT, KEYWORD_STRING, PUNCTUATION_RPAREN. Contexto: ParametrosOpc
Parsing error
```

Figure 12: Output Test 10.

Test 11: Conditional without block.

```
int main() {
    int a = 1;
    int x = 0;
    if (a == 1) x = 1;
    return x;
}
```

The grammar requires a block after the condition. When the parser looks for *Block* and finds the identifier `x`, the cell is empty and the error is reported in line 4. This requirement is what avoids the dangling-else ambiguity.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Error sintactico en linea 4: se encontro 'x' (TOKEN_IDENTIFIER). Se esperaba: PUNCTUATION_LBRA
CE. Contexto: Bloque
Parsing error
```

Figure 13: Output Test 11.

Test 12: Missing expression.

```
int x = ;
```

After the assignment operator the parser expands *Assignment*, and none of its productions starts with a semicolon, so the cell is empty.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Error sintactico en linea 1: se encontro ';' (PUNCTUATION_SEMICOLON). Se esperaba: OPERATOR_MI
NUS, OPERATOR_PLUS, PUNCTUATION_LPAREN, TOKEN_CONSTANT, TOKEN_IDENTIFIER, TOKEN_LITERAL. Conte
xto: Asignacion
Parsing error
```

Figure 14: Output Test 12.

4.5 Lexical errors

Test 13: Identifier that starts with a digit.

```
int 1x = 5;
```

The lexer classifies `1x` as an invalid identifier that starts with a digit. Since the list of tokens has an error token, the process ends before the parser is executed, which is the reason why a lexical error does not produce a syntactic or a semantic message.

```
[javi@Javi-Arch src]$ ./main

========== LEXICAL ANALYSIS ==========
Error lexico en linea 1: ERROR_ID_START_DIGIT ('1x')
Analisis lexico incorrecto
```

Figure 15: Output Test 13.

4.6 Summary of results

Test	What it verifies	Result
1	Official example	Accepted
2	Main constructs of the language	Accepted
3	Operator precedence	Accepted
4	Nested conditionals, no ambiguity	Accepted
5	Empty program	Accepted
6	Same name in different scopes	Accepted (hidden in the inner scope)
7	Repeated global declaration	Semantic error, line 2
8	Type incompatibility in initialization	Semantic error, line 1
9	Missing semicolon	Syntax error, line 3
10	Missing parenthesis	Syntax error, line 1
11	Conditional without block	Syntax error, line 4
12	Missing expression	Syntax error, line 1
13	Identifier that starts with a digit	Lexical error, line 1

Table 9: Summary of the tests.

5 Conclusions

The development showed that a grammar written in a natural way is not enough to build a predictive parser: it had to be transformed with the elimination of left recursion and with left factoring before FIRST and FOLLOW could fill a table without conflicts. The results confirm it. With a single token of lookahead the parser chooses the production of every non-terminal in all the tests, accepts the correct programs and stops in the first token that cannot continue a valid one, which explains why some errors are reported in the line that follows the mistake.

The tests also show where the grammar stops being enough. A repeated declaration is a valid sentence for the grammar, and it is the symbol table with its scopes that rejects it; the same name in an inner scope is accepted because it only hides the outer one. This is the difference between a syntax error, where the tree cannot be built, and a semantic error, where the tree exists and its meaning is invalid. In the same way, lexical errors are stopped before the parser because the parser only works on valid tokens.

The parse tree and the symbol table are the base of the semantic analysis. The tree keeps the lexemes and lines through the index of the tokens, and the table stores the type of every variable and parameter, which allows undeclared identifiers and incompatible types in initializations to be detected. As future work, the same lookup and type inference can be applied to assignments, conditions, call arguments and `return` statements, and functions can be registered.

A Names of the non-terminals

The program uses Spanish names for the non-terminals. The report uses the English names of the following table.

Program	**Report**	**Program**	**Report**
`Programa`	Program	`Comparacion`	Comparison
`ElementoGlobal`	GlobalElement	`ComparacionOpc`	OptionalComparison
`ContinuacionGlobal`	GlobalContinuation	`OperadorComparacion`	ComparisonOperator
`Declaracion`	Declaration	`ListaComparaciones`	ConditionList
`ListaVariables`	VariableList	`ListaComparacionesPrima`	ConditionListPrime
`ListaVariablesPrima`	VariableListPrime	`Conjuncion`	Conjunction
`Variable`	Variable	`ConjuncionPrima`	ConjunctionPrime
`InicializacionOpc`	OptionalInitialization	`ComparacionLogica`	LogicalComparison
`ListaArgumentos`	ArgumentList	`Bloque`	Block
`ListaArgumentosPrima`	ArgumentListPrime	`ListaSentencias`	StatementList
`ParametrosOpc`	OptionalParameters	`Sentencia`	Statement
`ListaParametros`	ParameterList	`SentenciaCondicional`	IfStatement
`ListaParametrosPrima`	ParameterListPrime	`ElseOpcional`	OptionalElse
`Parametro`	Parameter	`SentenciaFor`	ForStatement
`Tipo`	Type	`InicializacionFor`	ForInitialization
`TipoFuncion`	FunctionType	`ActualizacionFor`	ForUpdate
`Expresion`	Expression	`SentenciaWhile`	WhileStatement
`ExpresionPrima`	ExpressionPrime	`DefinicionFuncion`	FunctionDefinition
`Termino`	Term	`SentenciaRetorno`	ReturnStatement
`TerminoPrima`	TermPrime	`Asignacion`	Assignment
`Unario`	Unary	`ContinuarAsignacion`	ContinueAssignment
`Primario`	Primary	`ContinuarExpresion`	ContinueExpression
`LlamadaFuncionPrima`	FunctionCallPrime	`InicioSinId`	StartWithoutId

Table 10: Correspondence between the names of the program and of the report.

B Productions of the grammar

The productions keep the numbering of the program, which is the one printed in the trace. The terminals `ID`, `CONST` and `LIT` stand for identifiers, constants and literals.

1 *VariableList* → *Variable VariableListPrime*
2 *VariableListPrime* → `,` *Variable VariableListPrime*
3 *VariableListPrime* → ε
4 *Variable* → `ID` *OptionalInitialization*
5 *OptionalInitialization* → `=` *Assignment*
6 *OptionalInitialization* → ε

7 *ArgumentList* → *Expression ArgumentListPrime*
8 *ArgumentList* → ε
9 *ArgumentListPrime* → `,` *Expression ArgumentListPrime*
10 *ArgumentListPrime* → ε
11 *OptionalParameters* → *ParameterList*
12 *OptionalParameters* → ε
13 *ParameterList* → *Parameter ParameterListPrime*
14 *ParameterListPrime* → `,` *Parameter ParameterListPrime*
15 *ParameterListPrime* → ε
16 *Parameter* → *Type* `ID`
17 *Type* → `int`
18 *Type* → `float`
19 *Type* → `char`
20 *Type* → `string`
21 *FunctionType* → *Type*
22 *FunctionType* → `void`
23 *Expression* → *Term ExpressionPrime*
24 *ExpressionPrime* → `+` *Term ExpressionPrime*
25 *ExpressionPrime* → `-` *Term ExpressionPrime*
26 *ExpressionPrime* → ε
27 *Term* → *Unary TermPrime*
28 *TermPrime* → `*` *Unary TermPrime*
29 *TermPrime* → `/` *Unary TermPrime*
30 *TermPrime* → ε
31 *Unary* → `+` *Unary*
32 *Unary* → `-` *Unary*
33 *Unary* → *Primary*
34 *Primary* → `CONST`
35 *Primary* → `LIT`
36 *Primary* → `ID` *FunctionCallPrime*
37 *Primary* → `(` *Expression* `)`
38 *FunctionCallPrime* → `(` *ArgumentList* `)`
39 *FunctionCallPrime* → ε
40 *Declaration* → *Type VariableList* `;`
41 *Assignment* → `ID` *ContinueAssignment*
42 *Assignment* → *StartWithoutId ContinueExpression*
43 *ContinueAssignment* → `=` *Assignment*
44 *ContinueAssignment* → *FunctionCallPrime ContinueExpression*
45 *ContinueExpression* → *TermPrime ExpressionPrime*
46 *StartWithoutId* → `CONST`
47 *StartWithoutId* → `LIT`
48 *StartWithoutId* → `(` *Expression* `)`
49 *StartWithoutId* → `+` *Unary*
50 *StartWithoutId* → `-` *Unary*
51 *Comparison* → *Expression OptionalComparison*
52 *OptionalComparison* → *ComparisonOperator Expression*
53 *OptionalComparison* → ε

54 *ComparisonOperator* → `==`
55 *ComparisonOperator* → `>=`
56 *ComparisonOperator* → `<=`
57 *ComparisonOperator* → `>`
58 *ComparisonOperator* → `<`
59 *ComparisonOperator* → `!=`
60 *ConditionList* → *Conjunction ConditionListPrime*
61 *ConditionListPrime* → `||` *Conjunction ConditionListPrime*
62 *ConditionListPrime* → ε
63 *Conjunction* → *LogicalComparison ConjunctionPrime*
64 *ConjunctionPrime* → `&&` *LogicalComparison ConjunctionPrime*
65 *ConjunctionPrime* → ε
66 *LogicalComparison* → *Comparison*
67 *LogicalComparison* → `!` `(` *Comparison* `)`
68 *Block* → `{` *StatementList* `}`
69 *StatementList* → *Statement StatementList*
70 *StatementList* → ε
71 *Statement* → *Declaration*
72 *Statement* → *Assignment* `;`
73 *Statement* → *ReturnStatement*
74 *Statement* → *IfStatement*
75 *Statement* → *ForStatement*
76 *Statement* → *WhileStatement*
77 *Statement* → *Block*
78 *Statement* → `;`
79 *IfStatement* → `if` `(` *ConditionList* `)` *Block OptionalElse*
80 *OptionalElse* → `else` *Block*
81 *OptionalElse* → ε
82 *ForStatement* → `for` `(` *ForInitialization* `;` *ConditionList* `;` *ForUpdate* `)` *Block*
83 *ForInitialization* → *Type VariableList*
84 *ForInitialization* → *Assignment*
85 *ForInitialization* → ε
86 *ForUpdate* → *Assignment*
87 *WhileStatement* → `while` `(` *ConditionList* `)` *Block*
88 *FunctionDefinition* → *FunctionType* `ID` `(` *OptionalParameters* `)` *Block*
89 *ReturnStatement* → `return` *Assignment* `;`
90 *Program* → *GlobalElement Program*
91 *Program* → ε
92 *GlobalElement* → *Type* `ID` *GlobalContinuation*
93 *GlobalElement* → `void` `ID` `(` *OptionalParameters* `)` *Block*
94 *GlobalContinuation* → `(` *OptionalParameters* `)` *Block*
95 *GlobalContinuation* → *OptionalInitialization VariableListPrime* `;`

References

[1] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Boston, MA, USA: Pearson Addison-Wesley, 2006.